

Loyola University Chicago
Department of Computer Science

COMP 271: Data Structures I
Quiz 1 Practice Exam (Weeks 1–4)
Suggested time: 50 minutes

September 25, 2026

Full Name: _____



This is a practice exam. It is not graded. It covers the same material as Quiz 1 (Weeks 1 to 4: refactoring, type annotations, classes and special methods, encapsulation and properties, inheritance, polymorphism, abstraction, and the Programmer's Pact), and it is about twice as long.

To get the most out of it, work under real conditions: closed book, no notes, no devices, and a timer set to 50 minutes. Then check your answers against the solutions and review the topics you missed.

For coding questions, write Python that follows our course style (the Programmer's Pact): type hints, a one-line docstring, named constants instead of magic values, no `break/continue`, and one `return` per function.

1. **Refactoring.** Which statement best describes the goal of refactoring?

- ☐ Adding new features to a program as quickly as possible.
- ☐ Improving the structure of the code (less duplication, clearer names, behavior driven by data) while keeping its output the same.
- ☐ Rewriting a program in fewer lines, even if the output changes slightly.
- ☐ Replacing every loop with a separate function.

2. **Dynamic vs. strong typing.** Consider:

```
1 x = 5
2 x = "five"
3 y = "3" + 4
```

Which statement is true?

- ☐ Line 2 raises an error, because `x` was already an `int`.
- ☐ Line 3 sets `y` to `"34"`.
- ☐ Line 3 sets `y` to 7.
- ☐ Line 2 is fine (Python is dynamically typed), but line 3 raises a `TypeError` (Python is strongly typed).

3. **Type annotations.** What happens when this runs?

```
1 def square(n: int) -> int:
2     """Return n times n."""
3     return n * n
4
5 print(square(2.5))
```

- ☐ A `TypeError`, because 2.5 is not an `int`.
- ☐ It prints 6, because the result is converted to `int`.
- ☐ It prints 6.25; annotations are hints and are not enforced when the program runs.
- ☐ A `SyntaxError`, because annotations cannot be used with floats.

4. Class variables vs. instance variables.

```
1 class Robot:
2     _count = 0
3
4     def __init__(self, name: str) -> None:
5         self._name = name
6         Robot._count += 1
7
8 Robot("Ada")
9 Robot("Bolt")
10 Robot("Cog")
11 print(Robot._count)
```

(a) What is printed?

- ☐ 0
- ☐ 1
- ☐ 3
- ☐ An error, because `_count` is private.

(b) Suppose line 6 is changed to `self._count += 1`. What is printed now?

- ☐ 0
- ☐ 1
- ☐ 3
- ☐ An error, because `self` has no `_count`.

5. `__str__` and `__repr__`.

```
1 class Point:
2     def __init__(self, x: int, y: int) -> None:
3         self.__x = x
4         self.__y = y
5
6     def __str__(self) -> str:
7         return f"({self.__x}, {self.__y})"
8
9 p = Point(1, 2)
10 print(p)
11 print([p])
```

What do the two `print` calls show?

- ☐ (1, 2) and [(1, 2)]
- ☐ (1, 2) and something like [`<__main__.Point object at 0x...>`]
- ☐ Both show something like `<__main__.Point object at 0x...>`
- ☐ An error, because `__repr__` is missing.

6. `__eq__`. Point now has this method (and `key()` returns `(x, y)`):

```
1 def __eq__(self, other: object) -> bool:
2     """Compare two points by their coordinates."""
3     result = NotImplemented
4     if isinstance(other, Point):
5         result = self.key() == other.key()
6     return result
```

What does `Point(1, 2) == "hello"` evaluate to?

- ☐ An error, because `NotImplemented` is not a boolean.
- ☐ `NotImplemented`
- ☐ `False`
- ☐ `True`

7. **Sorting objects.** `songs` is a list `[Song]`. Which special method must `Song` define so that `sorted(songs)` works?

- ☐ `__eq__`
- ☐ `__lt__`
- ☐ `__len__`
- ☐ `__str__`

8. **UML relationships.** A **Team** keeps a list of **Player** objects. Players exist on their own: a player can leave one team and join another, and deleting a team does not delete its players. Which UML relationship fits best?

- ☐ Inheritance
- ☐ Aggregation
- ☐ Composition
- ☐ Realization

9. **Method resolution order.**

```
1 class A:
2     def greet(self) -> str:
3         return "A"
4
5 class B(A):
6     def greet(self) -> str:
7         return "B"
8
9 class C(B):
10     pass
```

What is `C.mro()`, and what does `C().greet()` return?

- ☐ [A, B, C, object], and "A"
- ☐ [C, B, A, object], and "B"
- ☐ [C, A, B, object], and "A"
- ☐ [C, object], and an error because C has no `greet`

10. **The Programmer's Pact.** Both functions return `True` when `target` is in `items`. Which one follows the Pact?

Version 1

```
1 def has(items: list[str],
2     target: str) -> bool:
3     """Say if target is in items."""
4     for item in items:
5         if item == target:
6             return True
7     return False
```

Version 2

```
1 def has(items: list[str],
2     target: str) -> bool:
3     """Say if target is in items."""
4     found = False
5     i = 0
6     while i < len(items) and not found:
7         found = items[i] == target
8         i += 1
9     return found
```

- ☐ Version 1, because it is shorter.
- ☐ Version 2: the stopping condition is in the loop condition and there is a single `return`.
- ☐ Both, since they return the same result.
- ☐ Neither, because both use a loop.

11. **Single underscore.**

```
1 class Pet:
2     def __init__(self, name: str) -> None:
3         self._name = name
4
5 print(Pet("Rex")._name)
```

Which statement is true?

- ☐ It prints `Rex`. A single underscore only signals “internal use”; Python does not block access.
- ☐ It raises an `AttributeError`, because `_name` is private.
- ☐ It prints `None`.
- ☐ A single underscore makes `_name` a class variable.

12. Abstract base classes.

```
1 from abc import ABC, abstractmethod
2
3 class Report(ABC):
4     def header(self) -> str:
5         return "== Report =="
6
7     @abstractmethod
8     def render(self) -> str:
9         """Return the full report text."""
10
11 class SalesReport(Report):
12     def render(self) -> str:
13         return self.header() + " sales"
```

Which statement is true?

- ☐ An abstract class cannot contain any concrete method such as `header()`.
- ☐ `SalesReport` must also redefine `header()`.
- ☐ `SalesReport().render()` returns `"== Report == sales"`: the subclass inherits the concrete `header()`.
- ☐ `header()` runs automatically when a `SalesReport` is created.

13. Aliasing. What is printed?

```
1 a = [1, 2]
2 b = a
3 b.append(3)
4 print(a)
```

- ☐ `[1, 2]`
- ☐ `[1, 2, 3]`
- ☐ `[3]`
- ☐ An error

14. **None vs. an empty value.** A function looks up a user's email:

```
1 def find_email(username: str) -> str | None:
```

Why is returning `None` when the user does not exist better than returning `""`? How should the caller check the result?

15. **Properties vs. getter/setter methods.** A class `Account` currently has methods `get_owner()` and `set_owner(new_owner)`, used like this:

```
1 print(acct.get_owner())  
2 acct.set_owner("Maya")
```

(a) Rewrite the two lines as they would look if `owner` were a `@property` with a setter.

(b) Give one advantage of the `@property` style and one advantage of explicit `get_/set_` methods.

16. **Following the Pact.** List three ways this function breaks the Programmer's Pact, then rewrite it so it follows the Pact.

```
1 def shipping(weight):  
2     if weight > 20:  
3         return weight * 1.5 + 10  
4     return weight * 1.5
```

17. **Reading a UML class diagram.**

BankAccount
- __owner: str
- __balance: float
+ deposit(amount: float): None
+ balance(): float

- (a) What do the - and + signs mean?
- (b) Write the Python skeleton of this class: the class header, an `__init__` that sets both attributes, and the two method headers with type hints (bodies can be ...).

18. **Tracing `super()`.** What does this program print?

```
1  class Base:
2      def __init__(self) -> None:
3          print("Base init")
4
5  class Middle(Base):
6      def __init__(self) -> None:
7          print("Middle start")
8          super().__init__()
9          print("Middle end")
10
11 class Leaf(Middle):
12     def __init__(self) -> None:
13         super().__init__()
14         print("Leaf init")
15
16 Leaf()
```

19. **Refactor with one helper.** These three functions are almost identical:

```
1 def total_even(numbers: list[int]) -> int:
2     total = 0
3     for n in numbers:
4         if n % 2 == 0:
5             total += n
6     return total
7
8 def total_odd(numbers: list[int]) -> int:
9     total = 0
10    for n in numbers:
11        if n % 2 == 1:
12            total += n
13    return total
14
15 def total_multiples_of_three(numbers: list[int]) -> int:
16     total = 0
17     for n in numbers:
18         if n % 3 == 0:
19             total += n
20    return total
```

(a) Write one helper `total_matching(numbers: list[int], divisor: int, remainder: int) -> int` that returns the sum of the numbers `n` for which `n % divisor == remainder`.

(b) Rewrite `total_odd` as a one-line wrapper around your helper.

(c) Explain why adding `total_multiples_of_five` is now much cheaper than before.

20. **A class with special methods.** Write a class `Song` that:

- (a) Stores a `title`, an `artist` and a length in `seconds (int)` in **private** attributes set by `__init__`, and exposes each one through a read-only `@property`.
- (b) Defines `__str__` so that `print(Song("Echo", "Lumen", 185))` shows `Echo by Lumen (185s)`, and `__repr__` so that a list of songs prints the same way.
- (c) Defines `__eq__` so that two songs are equal when they have the same title and artist (the length does not matter), using a `key()` method, and returns `NotImplemented` when compared with something that is not a `Song`.
- (d) Defines `__lt__` so that `sorted(songs)` orders songs from shortest to longest.

21. **Abstraction, inheritance and polymorphism.** You are given:

```

1  from abc import ABC, abstractmethod
2
3  class PhonePlan(ABC):
4      """A phone plan with a name and a base price."""
5      _plans_created = 0
6
7      def __init__(self, name: str, base_price: float) -> None:
8          """Create a plan and count it."""
9          self.__name = name
10         self.__base_price = base_price
11         PhonePlan._plans_created += 1
12
13     @property
14     def name(self) -> str:
15         """Return the plan name."""
16         return self.__name
17
18     @property
19     def base_price(self) -> float:
20         """Return the base price."""
21         return self.__base_price
22
23     def describe(self) -> str:
24         """Return a one-line summary of the plan."""
25         return f"{self.name}: ${self.monthly_cost()} per month"
26
27     @abstractmethod
28     def monthly_cost(self) -> float:
29         """Return what the plan costs each month."""

```

(a) Write `BasicPlan(PhonePlan)`, whose monthly cost is just the base price.

(b) Write `UnlimitedPlan(PhonePlan)`. Its constructor also takes a `data_fee`, reuses the parent's constructor for the name and base price, and stores the fee in a private attribute. Its monthly cost is the base price plus the data fee.

- (c) Write `cheapest_plan(plans: list[PhonePlan]) -> PhonePlan | None` that returns the plan with the lowest monthly cost, or `None` if the list is empty. Do not check the type of any plan, and use a single `return`.

- (d) After running `BasicPlan("Basic", 30.0)` and `UnlimitedPlan("Max", 25.0, 20.0)`, what is `PhonePlan._plans_`? Why?

- (e) `describe()` is written once, in `PhonePlan`, yet it shows `Basic: $30.0 per month` for the first plan and `Max: $45.0 per month` for the second. Explain what makes this work.

This is a blank sheet for your use. Label which problem you are using it for.